Cardiff Metropolitan University

in collaboration with ICBT Campus

CIS6003 Advanced Programming

WRIT1 Coursework

Sunrise Dental Clinic

Appointment and Patient Management System

CIS6003 WRIT1 report

Student name: Dayansan

Student number: 219134076

Batch / intake: CIS6003 / 2026

September 2026

Word count: about 4,200 (body text from Introduction to Conclusion)

Declaration

I confirm that this report and the Sunrise Dental Clinic software are my own work for CIS6003. I cited the books and websites I actually used. I did not copy another student's project. GitHub link is in Section 7.

Signature: Dayansan Date: 2 September 2026

Contents

UML diagrams and screenshots from the running system are already inserted in this Word file.

1 Introduction

This is my CIS6003 WRIT1. The case is Sunrise Dental Clinic in Colombo. They were still writing names in a diary and adding the bill on a calculator, so two people could get the same dentist slot and the total was sometimes wrong. I had to make a Java system, test it, and put the code on GitHub (Sommerville, 2016). I used NetBeans and Tomcat on my laptop. First week I only got the WAR to deploy, which sounds small but the jakarta packages took a while.

The brief has six staff jobs and I kept those as the main thing. Login, register a walk-in (name, address, phone, dentist, treatment, date, time), search by appointment number, calculate/print bill, help, and exit. I put the data in MySQL because patients, visits and bills need to stay linked (Elmasri and Navathe, 2016). I did not use a text file.

I made a website, JSP + servlets on Tomcat 10, Java 17 (Eclipse Foundation, 2024; Horstmann, 2022). Some people do Swing. I used the browser so the same services can also run as REST under /api/, which is what Fielding (2000) talks about. I added a patient login, admin reports and a page to change dentists/fees. Those are extras. They are not instead of the six jobs.

Section 2 is how I used Git over five days. Section 3 is UML. Section 4 is the implementation. Section 5 is the patterns. Section 6 is tests. Site on my PC: http://localhost:8080/sunrise-clinic. Repo: https://github.com/Dayansank/sunrise-clinic.

2 Git commit and branching strategy (5 days)

The lecturer wanted this built over five days, not one zip at the end. I made a branch for each day, merged to main when that part ran. Commit messages are normal sentences like on GitHub (Chacon and Straub, 2014). db.properties is in .gitignore so the MySQL password is not on GitHub. Branches are named feature/....

2.1 Day 1 - foundation (branch: feature/foundation)

Day 1 is just the skeleton. Maven WAR, .gitignore, web.xml, then the SQL with tables, two staff logins and fn_next_appointment_no(). Then model classes for inheritance: Person, ClinicUser, Patient, StaffUser, AdminUser, ReceptionUser, Appointment, Bill, Dentist, Treatment. Nothing books yet on day 1, I only wanted Tomcat to load the empty app.

- Commit: started the maven war project so i can run this on tomcat
- Commit: added the mysql script with tables, sample dentists and the two staff logins
- Commit: created person, staff, patient and appointment classes (multilevel inheritance)

2.2 Day 2 - data layer and patterns (branch: feature/data-and-patterns)

Day 2 is DAOs and patterns. DBConnection singleton, helpers for password and phone, then the pattern package, then AppointmentService / BillingService / AuthService. Still no nice menu. I merge when compile works. Fowler (2002) is the DAO idea, SQL not in the JSP.

- Commit: db connection singleton and dao classes to talk to mysql
- Commit: helpers for password hashing, validation and json
- Commit: put the design patterns in for booking, billing, validation and notifications
- Commit: service layer for login, booking, bills and qr tickets

2.3 Day 3 - staff desk (branch: feature/staff-desk)

Day 3 is the six jobs on screen. Login, register, search, bill, help, logout, plus AuthFilter (Eclipse Foundation, 2024). After this day reception can actually work the desk. If this day failed I would not have started the patient pages on day 4.

- Commit: staff login, walk-in booking, search and print bill

2.4 Day 4 - portal, API and reports (branch: feature/portal-and-api)

Day 4 is patient booking + QR, REST under /api/, and admin reports with Chart.js. Same clinic, not a second project. Merge when a patient can book a free slot and GET /api/appointments/{number} returns JSON.

- Commit: patient portal so people can book online and get a qr ticket
- Commit: rest apis for login, appointments, bills and live slots
- Commit: reports page with charts for admin

2.5 Day 5 - roles, live catalogue, UI and tests (branch: feature/roles-ui-tests)

Day 5 split admin and reception menus. Admin can add staff but not delete themselves. Catalogue from MySQL. Homepage slider. Then JUnit and README, merge, push.

- Commit: split admin and reception, and admin can add or delete staff
- Commit: clinic catalogue reads from mysql so website shows db changes
- Commit: homepage ui with slider and the clinic look
- Commit: junit tests and a short readme on how to run it

Some days I did more than one commit because compile failed and I had to fix it. Table 1 is the plan. Commit sentences match GitHub.

How I actually ran Git on this PC: I stayed on a feature branch until that day's pages opened in Tomcat. Then I merged to main. I did not force-push as a habit. If a file was only for my MySQL password it stayed in .gitignore. Chacon and Straub (2014) is the book I used when I forgot how merge works. The empty future branches are still there so I do not dump unfinished SMS code onto main.

Day	Branch	What I finish	Then I merge
1	feature/foundation	WAR, SQL, model inheritance	main
2	feature/data-and-patterns	DAO, patterns, services	main
3	feature/staff-desk	Six staff jobs on JSP	main
4	feature/portal-and-api	Portal, REST, reports	main
5	feature/roles-ui-tests	Roles, catalogue, UI, JUnit	main

Table 1: Five-day Git branching plan

3 Task A - Design

I drew UML from the classes I was going to write. Same names as the code, Appointment, BillingService, APT-2026-0001. Fowler (2004) says keep the class drawing small. Larman (2005) also says dont dump every servlet on the class diagram.

3.1 Assumptions

The brief skips some clinic rules so I wrote these down and used them in the diagrams and tests.

- Website on Tomcat, not a desktop exe. After login the six jobs are on a menu.
- Reception and admin are different. Reception books and bills. Admin does reports, staff, dentists. Different menus.
- Patients can book online. Extra. Still a normal APT number.
- MySQL makes APT-2026-0001. Nobody types it.
- Same dentist cannot have two BOOKED visits at the same time. Cancelled does not block the slot.
- Bill = treatment + consultation fee from clinic_settings. One visit, one bill max.
- Cancelled visit cannot be billed. After a bill the visit is COMPLETED.
- Exit is logout. Login is not include on every oval.
- Passwords hashed. Help text depends who is logged in.

I also assumed Sunday is closed, hours 9.00 to 16.30 for slots, and the sample fee 1500 unless admin changes it.

3.2 Use case diagram (include and extend) and functions

Figure 1 is the use case diagram. Include is for stuff that always happens. Extend is for extra stuff that only happens sometimes (Fowler, 2004; Larman, 2005). I nearly put login as include on every oval, then I stopped because AuthFilter already kicks guests to login.jsp. So login is its own use case. Dashed <<include>> goes from the main oval to the bit that always runs. Dashed <<extend>> goes from the extra oval back to the main one.

Use Case Diagram - Sunrise Dental Clinic

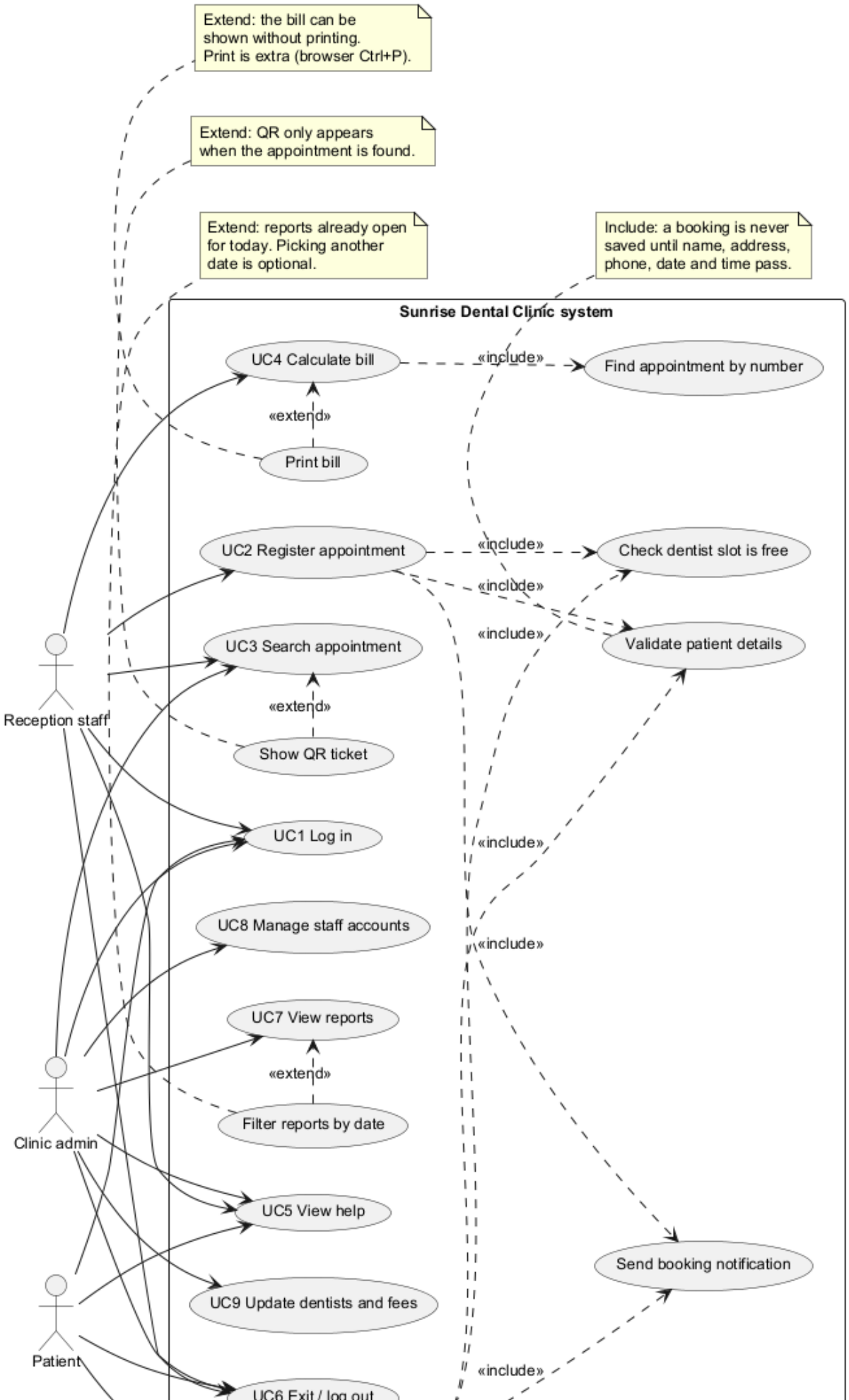

Figure 1: Use case diagram of Sunrise Dental Clinic, with <<include>> and <<extend>>

3.2.1 Actors

Reception staff is the front desk. They do UC1 login, UC2 register, UC3 search, UC4 bill, UC5 help, UC6 exit. They dont open reports. Booch, Rumbaugh and Jacobson (2005) say an actor is outside the system, so I drew three stick figures.

Clinic admin is office side. UC1, UC3, UC5, UC6, plus UC7 reports, UC8 staff accounts, UC9 dentists and fees. They dont do walk-in bills. That matches the Java StaffAccessPolicy.

Patient is the website user. UC1 patient login, UC5, UC6, UC10 book own appointment. They dont print the clinic invoice.

3.2.2 What each function does

UC1 Log in. Username and password. Staff from staff_users, patients from their own login. Right password and active account -> session. Wrong details stay on login.jsp.

UC2 Register appointment. Walk-in at the desk. Reception types name, address, phone, dentist, treatment, date, time. Gets a new APT-2026 number. This is instead of the paper book.

UC3 Search appointment. Type APT-2026-0001 (or whatever number). Shows patient, dentist, treatment, date, time, status. Or not found.

UC4 Calculate bill. Reception types the number. System loads the visit, treatment + fee, saves the bill, marks COMPLETED.

UC5 View help. Short steps. Reception gets register/search/bill. Admin gets reports/staff/dentists. Guests dont see this page.

UC6 Exit / log out. Kills the session. Staff go to the home page, patients go to patient login. Next person at the PC should not get the last bill page.

UC7 View reports. Admin page with charts and a list. Reception cannot open it. Todays date is enough.

UC8 Manage staff. Admin adds or deletes a login. Cannot delete yourself or the last admin. I found that out when I nearly deleted admin while testing.

UC9 Dentists and fees. Admin adds a dentist or changes the fee. After refresh the form and the bill use the new data. That is why I did not hardcode dentist names in Java.

UC10 Book own appointment. Patient picks dentist, treatment, date, free slot. Still a normal APT number. bookedBy = PATIENT.

Validate patient details (included). Always runs in UC2 and UC10. Name, address, phone, date, time go through BookingValidationChain. Fail = nothing saved. Include because you cannot book without this.

Check dentist slot is free (included). Also always in UC2 and UC10. isSlotTaken() plus a MySQL trigger. Include because double booking would mean it failed.

Find appointment by number (included). UC4 always loads the visit first. createBill() calls findByNumber(). Reception does not type the filling price by hand.

Send booking notification (included). After a good insert, onBooked() writes email and SMS rows. No skip button. I dont have Dialog/Mobitel connected, but the row is still written every time, so include.

Print bill (extends UC4). Bill page already has the three amounts. Print is Ctrl+P. They can look at the total and not print. If I used include it would say you must print to calculate, which is wrong.

Show QR ticket (extends UC3). Search is done when you find it or you dont. QR only if found.

Filter reports by date (extends UC7). Opening reports for today is enough. Changing the date is extra.

When I drew Figure 1 I kept moving the include arrows. First print was include on bill, then I changed it to extend because you can see the total without printing. Notification stayed include because the code always writes the row. QR only after a found search so extend.

3.3 Class diagram and functions

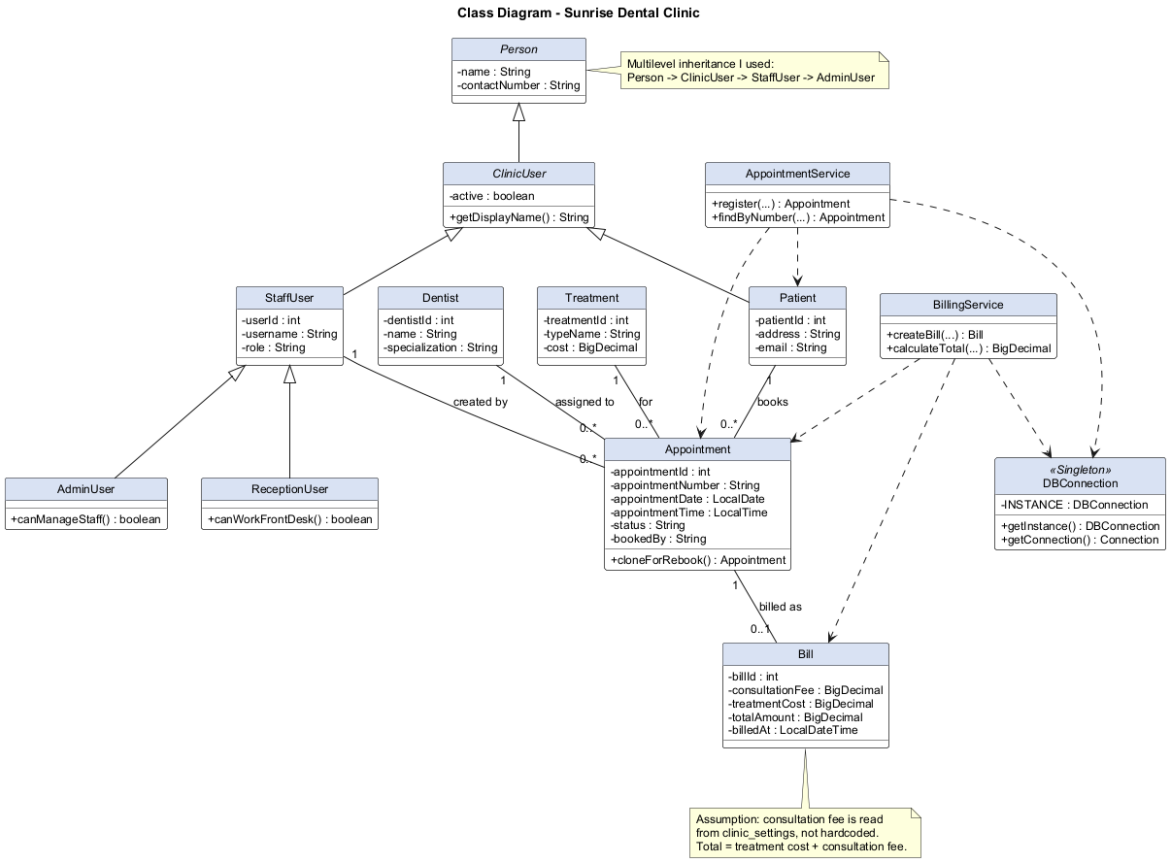

Figure 2: Class diagram (inheritance and main objects)

Figure 2 is the main Java classes. Inheritance is Person -> ClinicUser -> StaffUser -> AdminUser, same path for ReceptionUser. Horstmann (2022) has this kind of person tree. Appointment has the number, date, time, status. Bill has the three money fields. Dentist and Treatment come from MySQL. Services do the work, DBConnection is the singleton. MVC is JSP / servlet / service (Buschmann et al., 1996). I tried not to put SQL in the JSP, Martin (2009) is always going on about keeping classes small.

3.4 Sequence diagrams

Class diagram does not show time so I drew six sequences (Sommerville, 2016; Booch, Rumbaugh and Jacobson, 2005). Each picture then a short note under it. I did not draw staff-admin as a sequence, it is extra.

3.4.1 Sequence diagram - log in

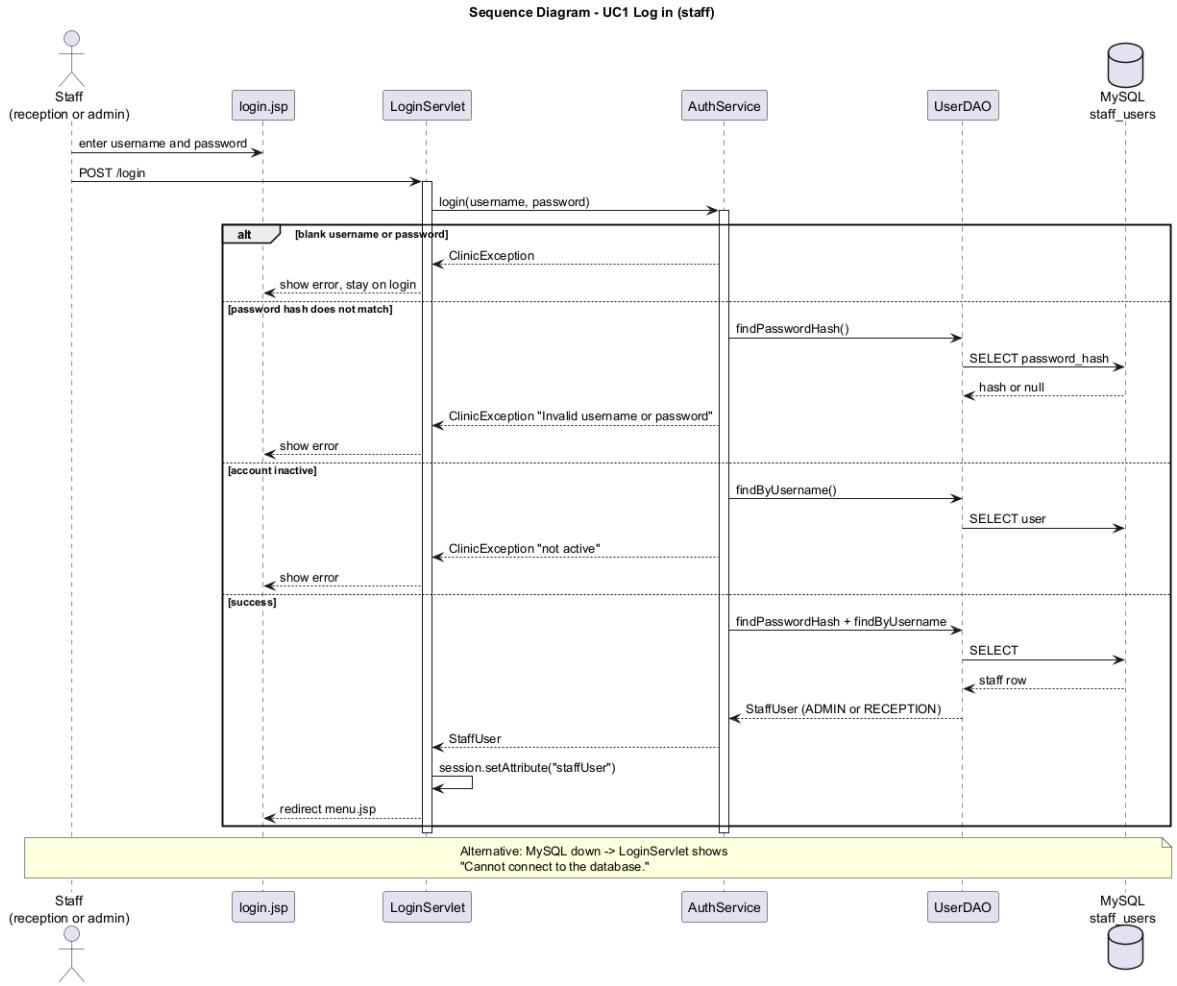

Figure 3: Sequence diagram for UC1 Log in

Reception or admin type username and password on login.jsp. LoginServlet -> AuthService.login() -> UserDAO, checks the hash. Blank, wrong hash, or inactive = stay on login. MySQL down = connection error. Success puts StaffUser in the session and goes to menu.jsp. Admin sees reports, reception sees register and bill. I tested wrong password a few times, it just stays on the same page with an error, no stack trace on the screen.

3.4.2 Sequence diagram - register appointment

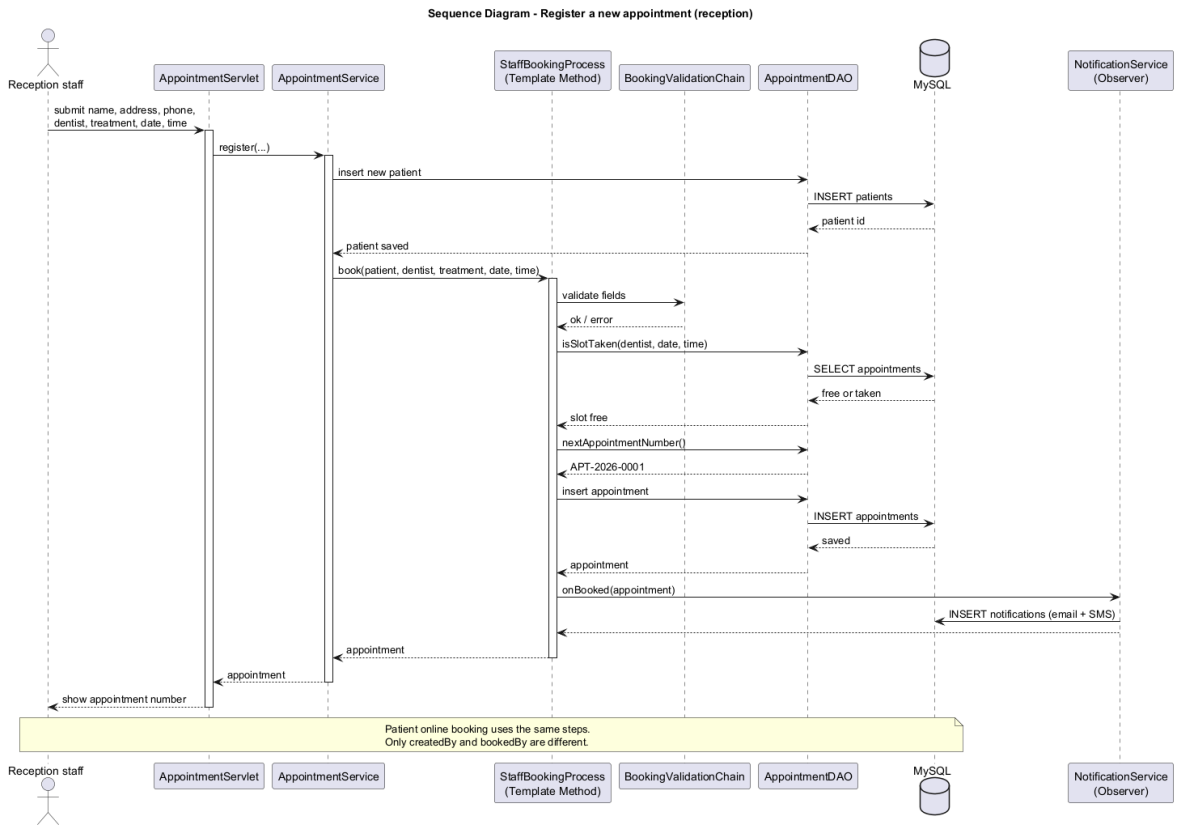

Figure 4: Sequence diagram for UC2 Register appointment

Reception submits the walk-in form. AppointmentServlet -> register(). Patient row first, then StaffBookingProcess: validate, check slot, next APT number, insert, notify. Page shows the number. If validation or slot fails, ClinicException and nothing saved. Online booking is PatientBookingProcess, same steps, different bookedBy. I tried booking two fillings for the same dentist at 10.30 and the second one was rejected, which is the whole point of the diary replacement.

3.4.3 Sequence diagram - search appointment

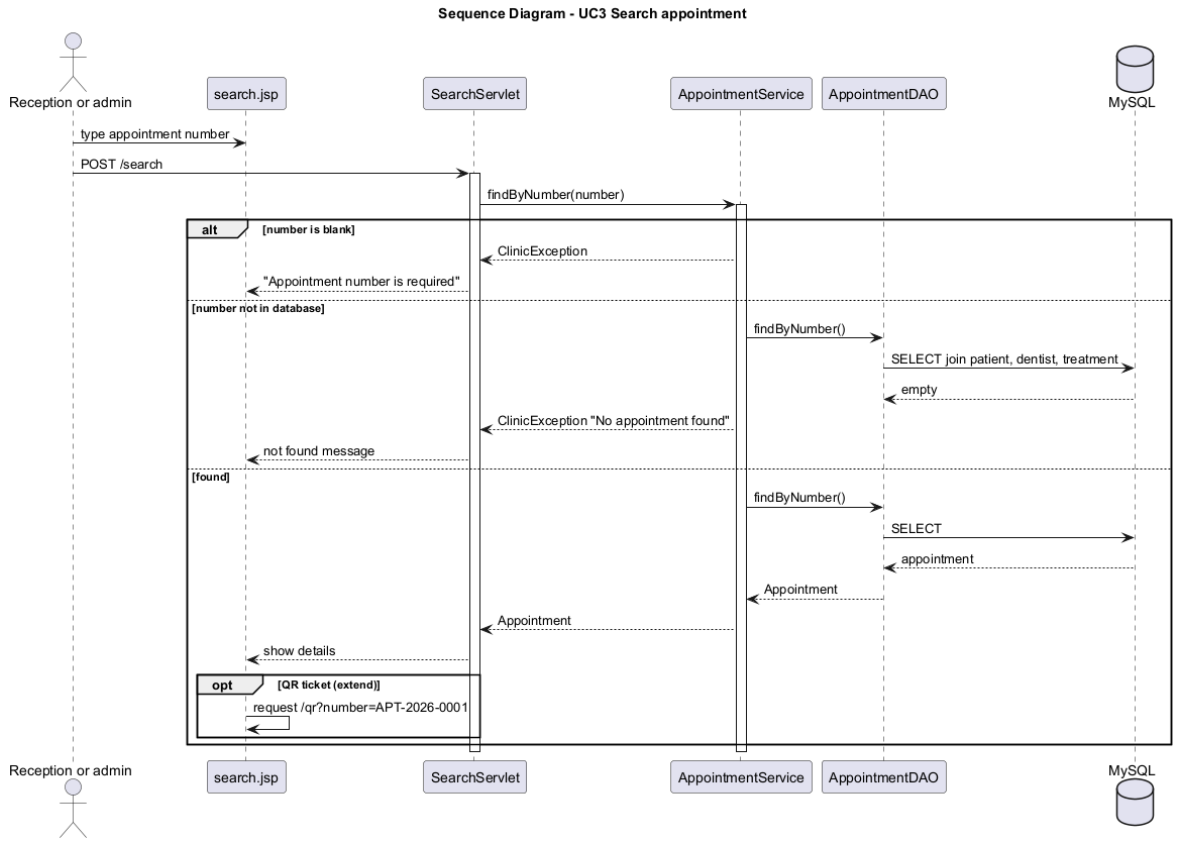

Figure 5: Sequence diagram for UC3 Search appointment

Staff type APT-2026-0001 on search.jsp. findByNumber(). Empty box = error. Unknown number = not found. If it exists you see the details. QR is extend, only after a hit. Failed search does not call QrServlet. I used APT-2026-9999 as a not-found test.

3.4.4 Sequence diagram - calculate bill

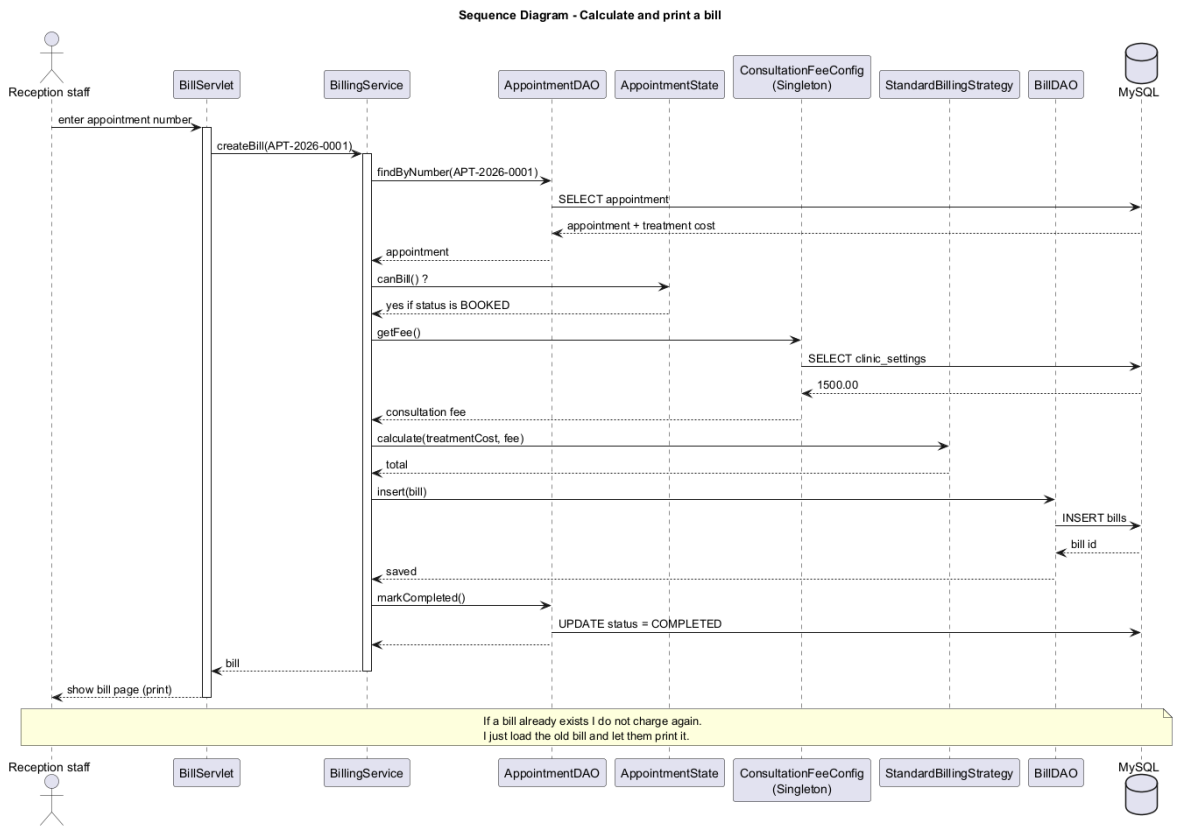

Figure 6: Sequence diagram for UC4 Calculate bill

Reception types the number on bill.jsp. createBill() finds the appointment first (include). Not BOOKED = State blocks it. Bill already there = show the old one. Else fee from ConsultationFeeConfig, strategy adds treatment + fee, save, mark COMPLETED. Print is extend, Ctrl+P after the amounts are on screen. Filling 8000 + fee 1500 was 9500 when I printed it.

3.4.5 Sequence diagram - view help

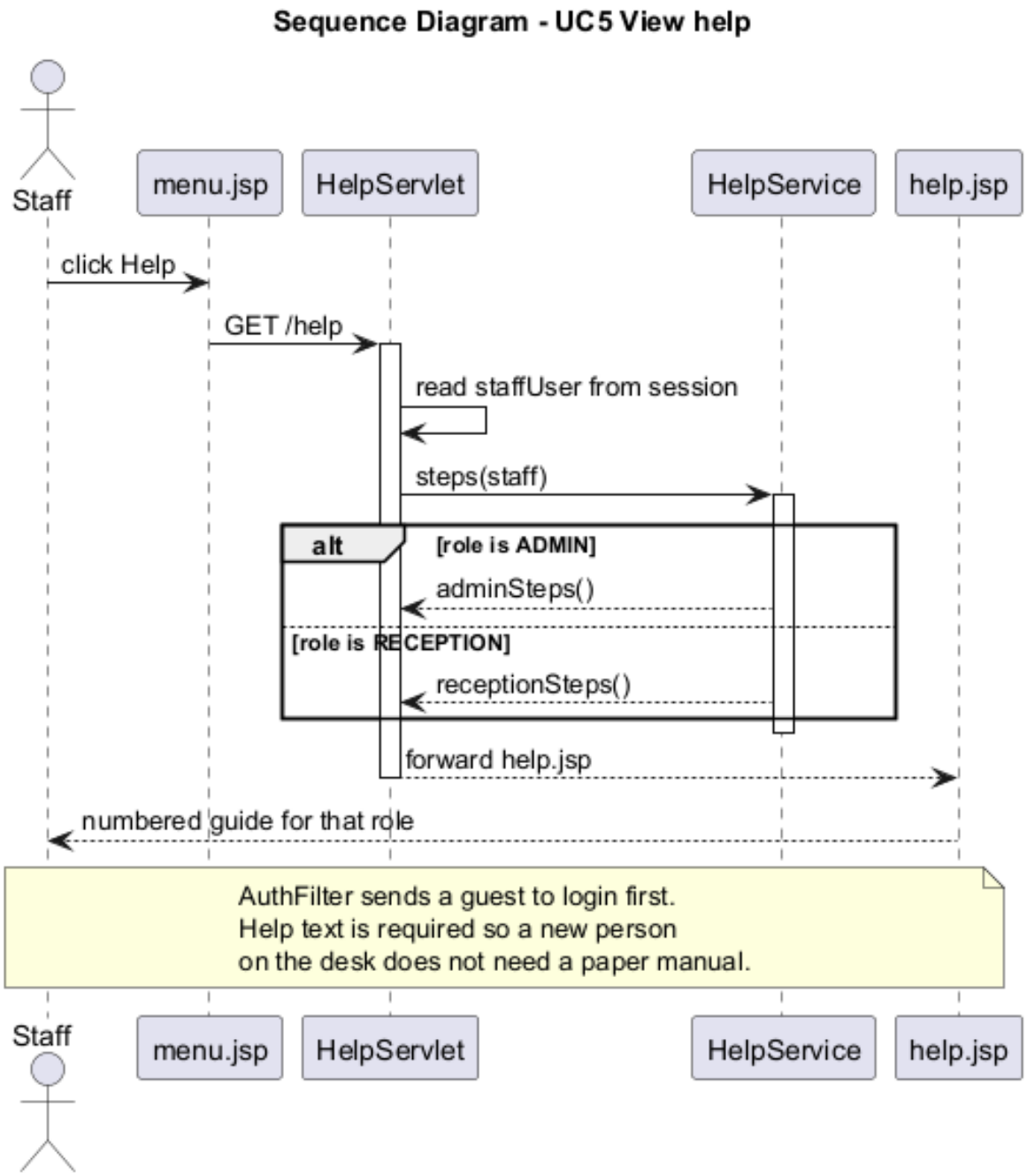

Figure 7: Sequence diagram for UC5 View help

Click Help. HelpServlet looks at the role. Reception gets walk-in/bill steps, admin gets staff/reports. No extra table. AuthFilter stops guests. I copied the wrong help onto both roles at first, then split HelpService.

3.4.6 Sequence diagram - exit / log out

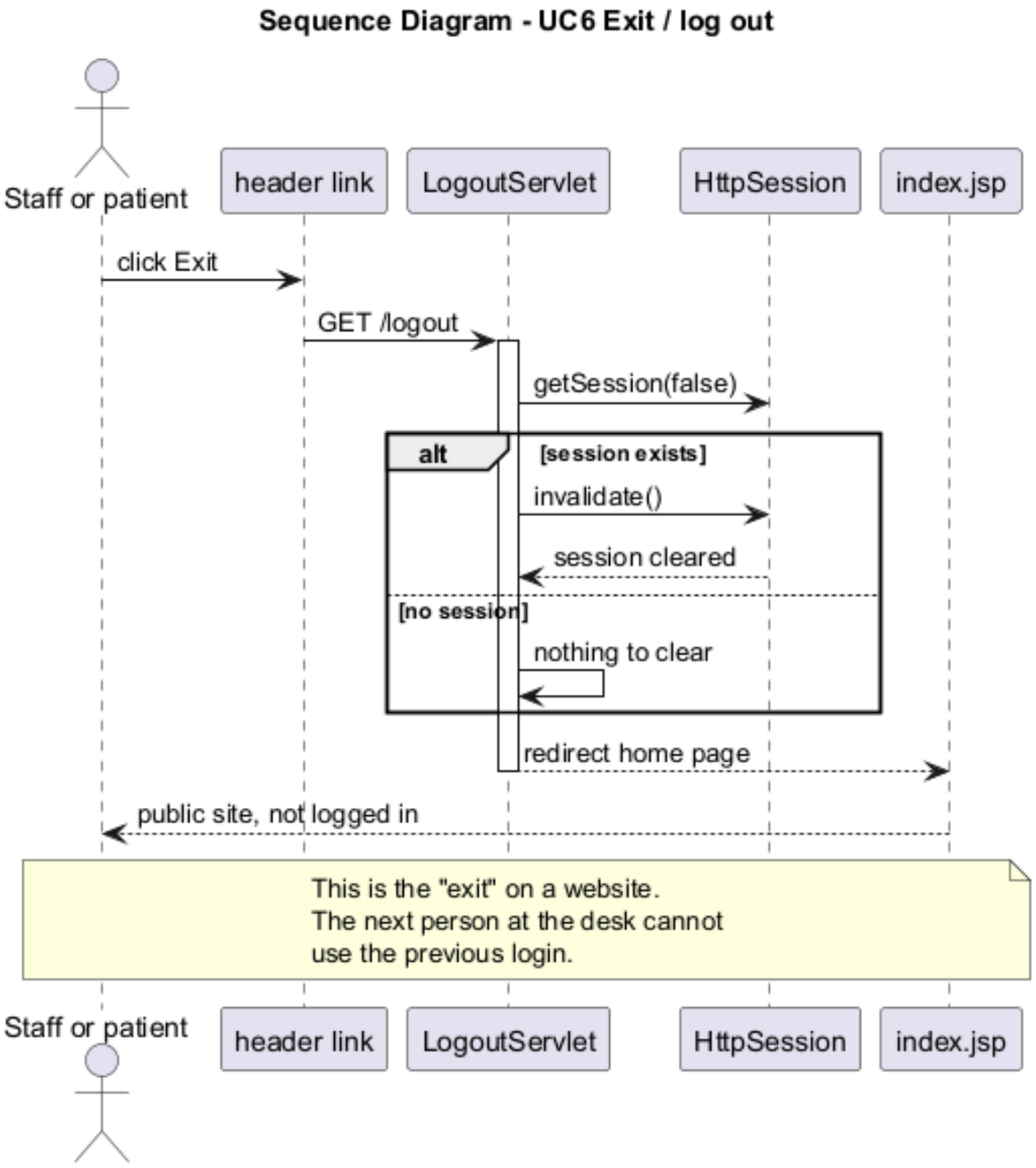

Figure 8: Sequence diagram for UC6 Exit

Click Exit. LogoutServlet kills the session if there is one and redirects. Staff -> home page, patients -> patient login. No MySQL write. Back button should not open the last bill because of AuthFilter. On a shared reception PC this matters more than closing a window.

4 Task B - Implementation

I coded Java 17, Maven WAR sunrise-clinic, NetBeans, Tomcat 10.1, MySQL 8 (Oracle, 2021; Apache Software Foundation, 2024; Oracle, 2024). URL is

http://localhost:8080/sunrise-clinic. Request goes browser -> servlet -> service -> DAO -> MySQL (Fowler, 2002). If I add a dentist in the catalogue it shows in the dropdown after I refresh.

On my laptop I import sunrise_clinic.sql once, set db.properties, then mvn package and copy the WAR into Tomcat webapps. Logins I used while writing this report: admin / Admin@123, reception / Staff@123, kamal@sunrise.lk / Patient@123. If Tomcat is already running I just refresh. The homepage text is normal clinic wording now, not a long advert. Staff login is a simple form. After reception login the six jobs are on the menu the same as the brief.

Screenshots below are from my Tomcat. Reception menu is Register, Search, Bill, Help, Exit. Admin is Reports, Staff, Catalogue, Search, Help, Exit. Search shows QR. Bill shows fee, treatment, total.

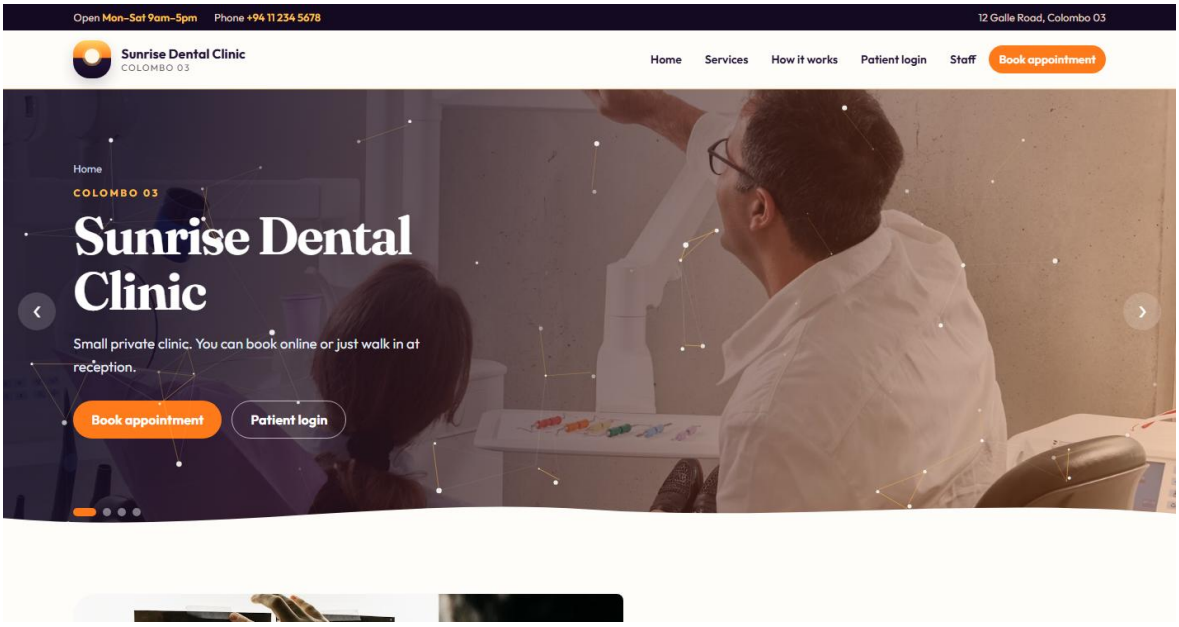

Figure 9: Public home page of Sunrise Dental Clinic

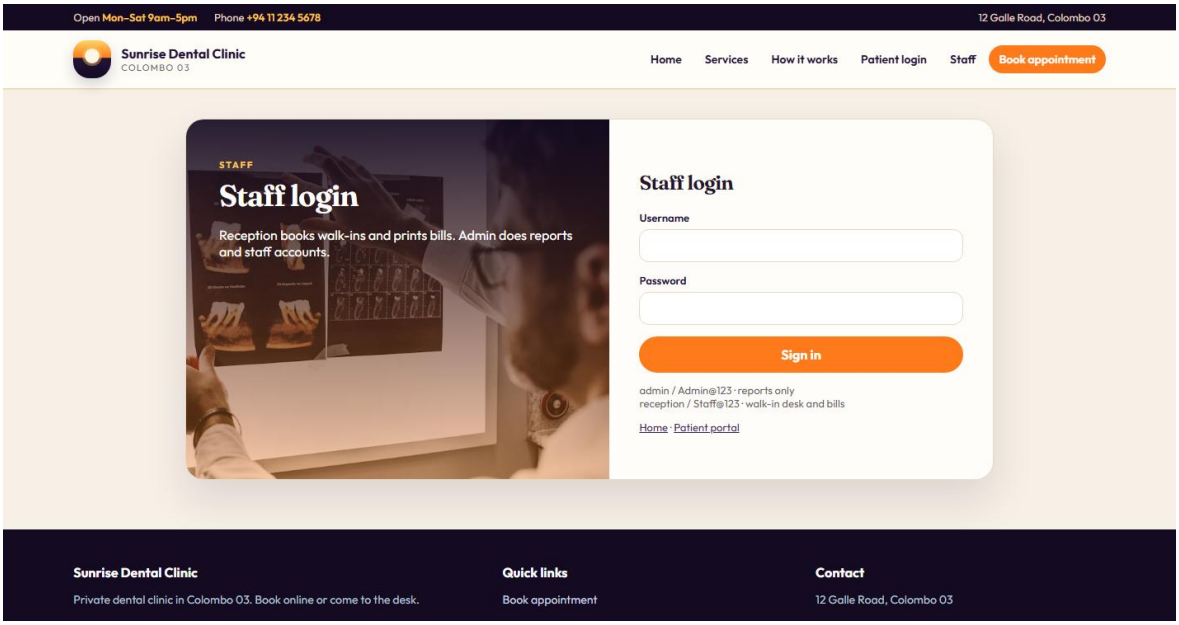

Figure 10: Staff login

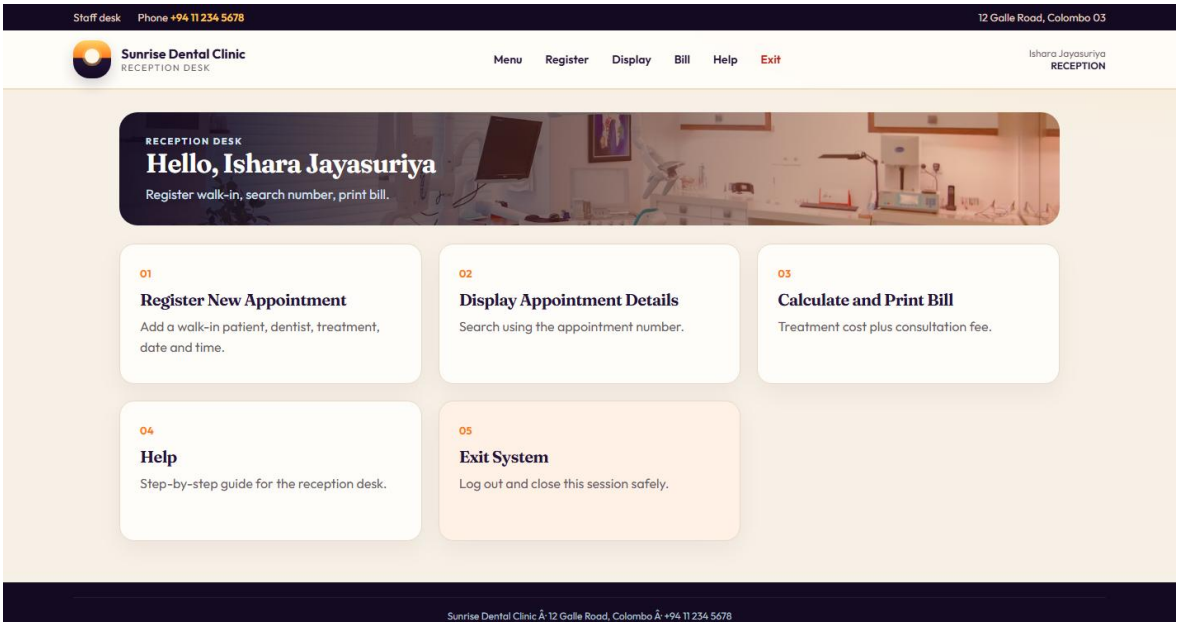

Figure 11: Reception menu after login

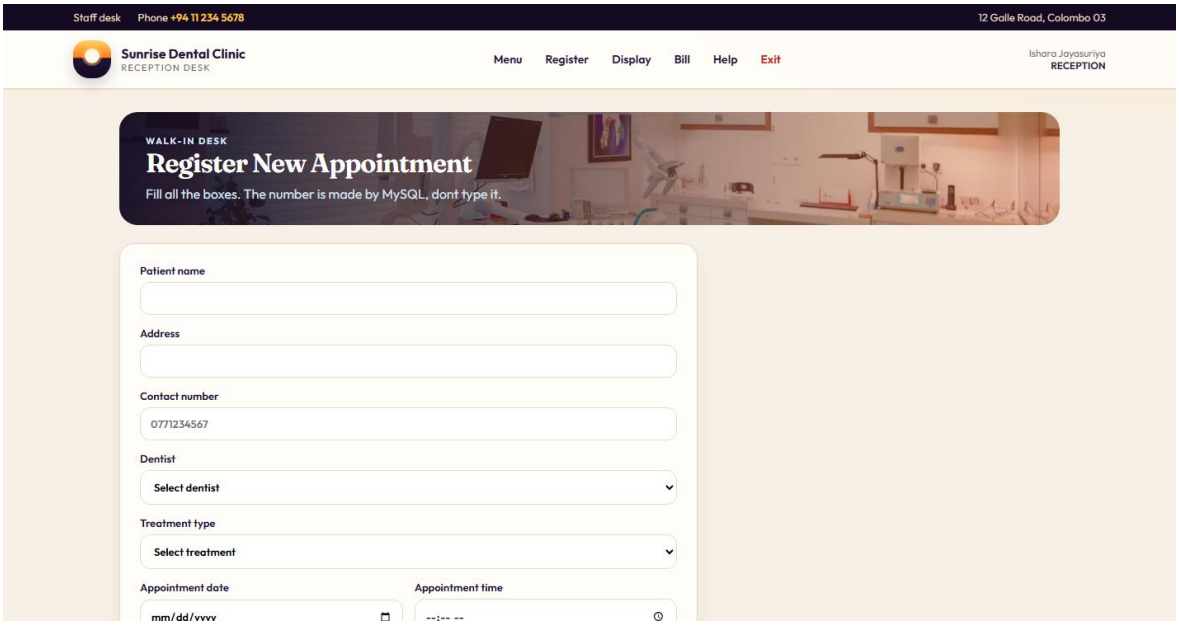

Figure 12: Register new appointment

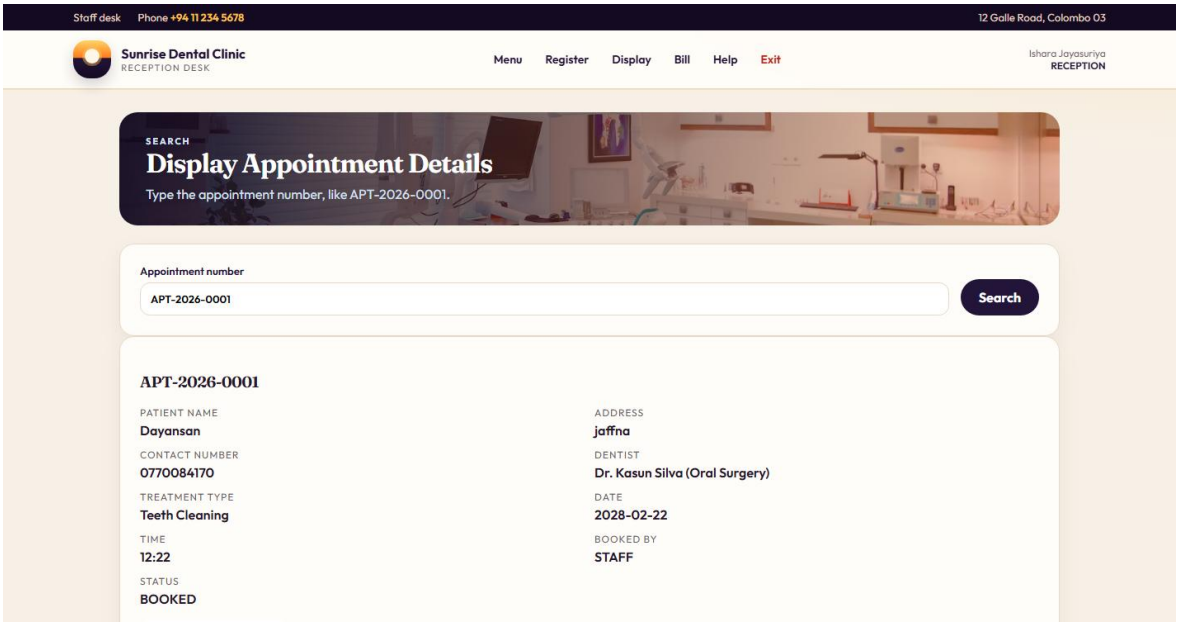

Figure 13: Search by appointment number with QR ticket

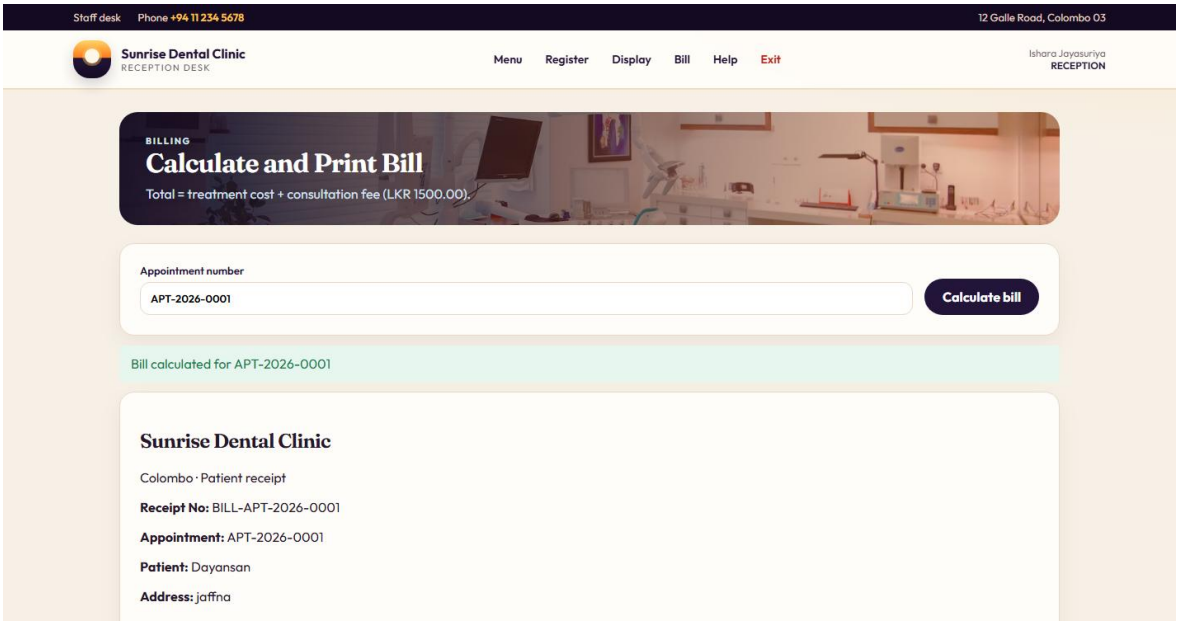

Figure 14: Calculate and print bill

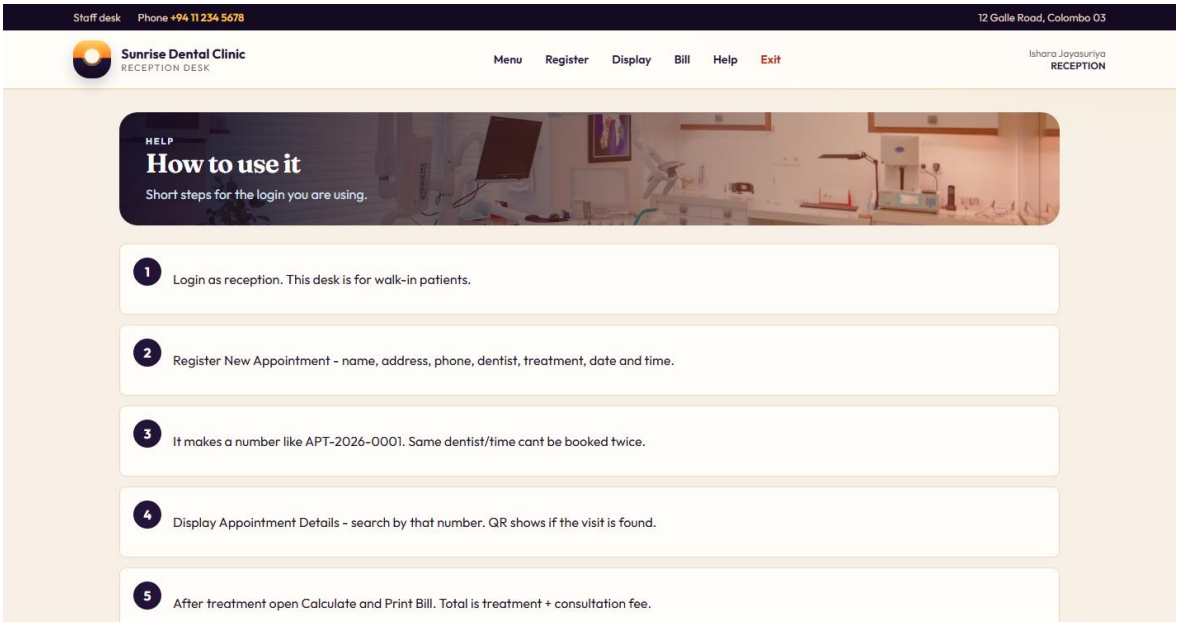

Figure 15: Reception help page

Problems I hit: first Tomcat 10 vs javax vs jakarta, I had to change imports. MySQL trigger blocked a booking Java had already checked, then I realised I was testing with a cancelled row still counted. Admin and reception sharing one menu looked wrong so I split it on day 5. Slider on the home page is only look, it does not book anyone.

Name, address, phone, date, time get checked before save. Phone is 10 digits starting 0, or +94. Java checks the slot and there is also a MySQL trigger. Tables are in

sunrise_clinic.sql (Elmasri and Navathe, 2016). Passwords are SHA-256 with a salt, OWASP Foundation (2021) says dont store plain passwords. Fee starts at LKR 1500.

REST is POST /api/auth/login, POST /api/appointments, GET /api/appointments/APT-2026-0001 and the bill/slot URLs (Fielding, 2000). I am running this on localhost only. Admin reports have three Chart.js charts. Reception cannot open that page.

5 Design patterns used in the Java code

I put patterns on booking and billing. Gamma et al. (1995) is the main book. Freeman et al. (2004) was easier when I was stuck on Observer. Package is com.sunrise.clinic.pattern. I will go through each one I actually coded.

5.1 Singleton

Singleton is one object for the whole app (Gamma et al., 1995). DBConnection reads db.properties once. If every DAO opened its own properties file I would forget to change the password in one of them. ConsultationFeeConfig.getInstance() reads the fee from clinic_settings so every bill uses the same number. If admin changes the fee on the catalogue page the next bill picks it up. QrCodeService is also one instance, it just makes PNG bytes. Test consultationFeeConfigIsSingleton() checks getInstance() twice and it is the same object.

5.2 Factory Method

I did not want AppointmentServlet calling new Appointment with a long constructor. AppointmentFactory fills dentist, treatment, date, time and BOOKED. The servlet just passes the form strings. If I add a field later I change the factory, not five servlets.

5.3 Abstract Factory

ClinicNotificationFactory makes the email sender and the SMS sender together. BookingTemplate asks the factory, it does not new EmailChannelAdapter itself. Right now both adapters only write a row in notifications. If I ever plug in a real SMS API I can swap the factory and leave the booking code.

5.4 Builder

QR text has a few bits, number, name, dentist, date, time. QrTicketBuilder chains those methods then build(). I mixed the order once with a constructor and the QR said the dentist was the patient. Builder made that harder to mess up. Test buildsReadableTicketText() looks for the APT number in the string.

5.5 Prototype

cloneForRebook() copies dentist and treatment from an old visit so the patient does not fill everything again. It is a new row, status goes back to BOOKED, new number later. I used this on the portal book-again button. Test prototypeCopiesDentistAndTreatment() checks those two fields.

5.6 Adapter

BookingTemplate only knows NotificationChannel.send(). EmailChannelAdapter and SmsChannelAdapter implement that and write to NotificationDAO. There is no real SMTP yet. I still wanted the booking code to call send() so later I can change only the adapter.

5.7 Decorator

QrConfirmationDecorator wraps BasicConfirmation and adds a QR line. I did not copy the whole confirmation class. Test decoratorAddsQrMessage() checks the extra line is there.

5.8 Facade

ZXing needs BitMatrix and a writer. I wrapped that in QrCodeService so search.jsp only asks for PNG bytes. generatesPngBytes() checks something actually comes back. If ZXing changes I only touch that one class.

5.9 Proxy

AuditedAppointmentService sits in front of AppointmentService on cancel. It still cancels, it also writes a log. CancelAppointmentCommand uses the proxy. I did not want log code sitting in the main service method.

5.10 Template Method

BookingTemplate.book() is final: validate, load dentist/treatment, check slot, save, notify. StaffBookingProcess and PatientBookingProcess only change createdBy. Figure 4 is the reception path. I did this because I kept forgetting the slot check when I copied code into the patient servlet.

5.11 Chain of Responsibility

NameHandler then AddressHandler then PhoneHandler then DateHandler then TimeHandler. Short name dies at the first one. Bad phone dies at PhoneHandler. That is the include Validate on Figure 1. Tests: rejectsShortNameAtFirstLevel, rejectsShortPhone, acceptsValidBookingData. Adding a new rule is another handler.

5.12 Observer

After insert, BookingTemplate calls onBooked() on whoever is in the list. NotificationService writes email/SMS rows. If I add reminders later I add another observer. Always runs after a successful save, so include not extend on Figure 1.

5.13 Strategy

StandardBillingStrategy just adds treatment + fee. Test is 8000 + 1500 = 9500. Figure 6 uses it. Menus are also strategy: AdminAccessPolicy vs ReceptionAccessPolicy. Reception cannot open /reports even if they type the URL. If the clinic changes the bill formula I add another BillingStrategy class.

5.14 Command

CancelAppointmentCommand has execute(). The portal servlet does not call five DAO methods. Right now execute() runs immediately. I might queue it later but not for this submission.

5.15 State

BOOKED can bill or cancel. COMPLETED cannot cancel. CANCELLED cannot bill. ExtraPatternsTest.stateBlocksCancelWhenCompleted() covers that. I had if (status.equals("COMPLETED")) in two places before, they got out of date, so I moved it to the enum.

5.16 MVC, DAO and multilevel inheritance

Not GoF, but I used them. JSP view, servlet controller, service/model (Buschmann et al., 1996). SQL only in DAO classes (Fowler, 2002). Person -> ClinicUser -> StaffUser -> AdminUser (Horstmann, 2022). Three JUnit tests for the inheritance. Patient is ClinicUser too, just no staff username.

6 Task C - Testing

I wrote JUnit 5 tests with Mockito, then I clicked the screens on Tomcat (JUnit Team, 2024). Table 2 has 45 cases, 37 @Test methods and 8 I did by hand. I did TDD for the bill first: 8000 + 1500 should be 9500, then I wrote StandardBillingStrategy. Same for a short phone 07712, then isValidPhone() (Beck, 2003). Mockito fakes DAOs so mvn test runs without MySQL. The manual ones need Tomcat up. Sommerville (2016) says do both, so I did.

A few tests failed the first time. Double booking one was because I forgot cancelled should not block the slot. Help text test failed when I still had the same help for admin and reception. After I split HelpService it passed.

ID	JUnit method / action	What I am testing	Input	Expected result	Type
TC01	AuthServiceTest.acceptsValidStaff	UC1 happy login	admin / Admin@123	StaffUser returned	JUnit
TC02	AuthServiceTest.rejectsWrongPassword	UC1 wrong password	admin / wrong	ClinicException	JUnit
TC03	AuthServiceTest.rejectsBlankUsernameOrPassword	UC1 blank fields	empty user or password	ClinicException	JUnit
TC04	PasswordUtilTest.hashesAdminPasswordToKnownValue	Password hash matches SQL seed	Admin@123	Same SHA-256 hash	JUnit
TC05	PasswordUtilTest.matchesCorrectPassword	Hash verify	correct password	matches() is true	JUnit
TC06	ValidationUtilTest.acceptsSriLankanPhoneNumbers	UC2 include validate phone	0771234567 and +94771234567	Accepted	JUnit
TC07	ValidationUtilTest.rejectsPastDatesAndSundayAndShortNames	UC2 include validate date/name	past date, Sunday, name Al	Rejected	JUnit

TC08	ValidationUtilTest.acceptsValidAppointmentInput	UC2 valid form fields	normal name, phone, future date	Accepted	JUnit
TC09	BookingValidationChainTest.rejectsShortNameAtFirstLevel	Chain stops on name	name Al	Error at NameHandler	JUnit
TC10	BookingValidationChainTest.rejectsShortPhone	Chain stops on phone	07712	Error at PhoneHandler	JUnit
TC11	BookingValidationChainTest.acceptsValidBookingData	Full validation chain	valid booking context	All handlers pass	JUnit
TC12	AppointmentServiceTest.rejectsDoubleBooking	UC2 include slot check	same dentist/date/time BOOKED	Rejected, no second row	JUnit
TC13	SlotServiceTest.clinicSlotsRunFromNineToHalfFour	Clinic opening hours	weekday	09:00 to 16:30 slots	JUnit
TC14	SlotServiceTest.hidesTakenSlot	Taken slot hidden on portal	already BOOKED time	That time not listed	JUnit
TC15	BillingServiceTest.createBillRejectsUnknownAppointment	UC4 include find	APT-2026-9999	ClinicException not found	JUnit
TC16	BillingPatternTest.standardStrategyAddsConsultationFee	UC4 bill math	8000 + 1500	Total 9500.00	JUnit
TC17	BillingPatternTest.consultationFeeConfigIsSingleton	Singleton fee config	two getInstance() calls	Same object	JUnit
TC18	ExtraPatternsTest.stateBlocksCancelWhenCompleted	State pattern	COMPLETED visit	canCancel() false	JUnit
TC19	HelpServiceTest.receptionHelpMentionsRegisterAndBill	UC5 reception help	RECEPTION role	Register and bill text	JUnit
TC20	HelpServiceTest.adminHelpCoversStaffCatalogueAndReports	UC5 admin help	ADMIN role	Staff, catalogue, reports	JUnit
TC21	StaffAccessPolicyTest.receptionHandlesDeskNotReports	Reception menu strategy	reception user	Desk yes, reports no	JUnit
TC22	StaffAccessPolicyTest.adminHandlesReportsNotDesk	Admin menu strategy	admin user	Reports yes, desk no	JUnit
TC23	StaffServiceTest.createsReceptionAccount	UC8 create staff	desk2 / reception	Row inserted	JUnit
TC24	StaffServiceTest.rejectsDuplicateUsername	UC8 duplicate user	username admin	Rejected	JUnit
TC25	StaffServiceTest.cannotDeleteOwnAccount	UC8 delete self	logged-in admin id	Rejected	JUnit
TC26	StaffServiceTest.deletesReceptionWithNoAppointments	UC8 delete unused	reception with 0 visits	Deleted	JUnit

		reception			
TC27	CatalogServiceTest.addsDentistToDatabase	UC9 add dentist	new dentist name	Row in dentists	JUnit
TC28	CatalogServiceTest.refusesDeleteWhenDentistHasAppointments	UC9 delete busy dentist	dentist with visits	Rejected	JUnit
TC29	CatalogServiceTest.savesConsultationFee	UC9 update fee	new fee amount	clinic_settings updated	JUnit
TC30	QrCodeServiceTest.generatesPngBytes	UC3 extend QR	appointment number	PNG bytes returned	JUnit
TC31	QrTicketBuilderTest.buildsReadableTicketText	Builder pattern	ticket fields	Text contains APT number	JUnit
TC32	ExtraPatternsTest.prototypeCopiesDentistAndTreatment	Prototype rebook	cloneForRebook()	Dentist and treatment copied	JUnit
TC33	ExtraPatternsTest.decoratorAddsQrMessage	Decorator confirmation	BasicConfirmation wrap	QR line added	JUnit
TC34	ReportChartServiceTest.doughnutCountsBookedVisits	UC7 doughnut data	list with BOOKED rows	Count matches	JUnit
TC35	MultilevelInheritanceTest.adminUserIsMultilevelStaff	Inheritance	new AdminUser()	is StaffUser and Person	JUnit
TC36	MultilevelInheritanceTest.receptionUserIsMultilevelStaff	Inheritance	new ReceptionUser()	is StaffUser and Person	JUnit
TC37	MultilevelInheritanceTest.patientIsClinicUser	Inheritance	new Patient()	is ClinicUser	JUnit
TC38	Staff login on Tomcat	UC1 reception menu	reception / Staff@123	Register and Bill links	Manual
TC39	Register walk-in on Tomcat	UC2 save visit	Kamal, 0771234567, Filling, tomorrow 10:30	APT-2026-xxxx shown	Manual
TC40	Search found + QR on Tomcat	UC3 and QR extend	APT-2026-0001	Details and QR image	Manual
TC41	Print bill on Tomcat	UC4 print extend	bill page on screen	Browser print works	Manual
TC42	Exit then Back button	UC6 session ended	click Exit, then Back	Protected page blocked	Manual
TC43	Patient portal book	UC10	kamal@sunrise.lk books a free slot	APT number, bookedBy PATIENT	Manual
TC44	REST get appointment	Web service	GET /api/appointments/APT-2026-0001	JSON visit	Manual
TC4	Reports date filter	UC7 date	admin picks another	Table and	Manua

5		extend	date	charts change	l

Table 2: Forty-five test cases (37 JUnit methods plus 8 manual Tomcat checks)

There are 17 test classes, 37 @Test methods. Sample data is Nadeesha (admin), Ishara (reception), Kamal at 12 Galle Road, Filling 8000, fee 1500. I used that so 8000+1500 stays 9500. I run `mvn test` from the project folder. If MySQL is down the JUnit ones still pass because of mocks. Manual TC38 to TC45 need Tomcat started.

I also tried reception opening /reports by typing the URL. It blocked. Patient kamal@sunrise.lk could book a slot that reception had not taken. GET /api/appointments/APT-2026-0001 gave JSON. That is TC44.

7 Task D - GitHub

Public repo: https://github.com/Dayansank/sunrise-clinic

I pushed main after the five days. History is small commits, not one giant commit. First one is started the maven war project so i can run this on tomcat (Chacon and Straub, 2014). I did not put db.properties on GitHub. If someone clones they copy the file and put their own password.

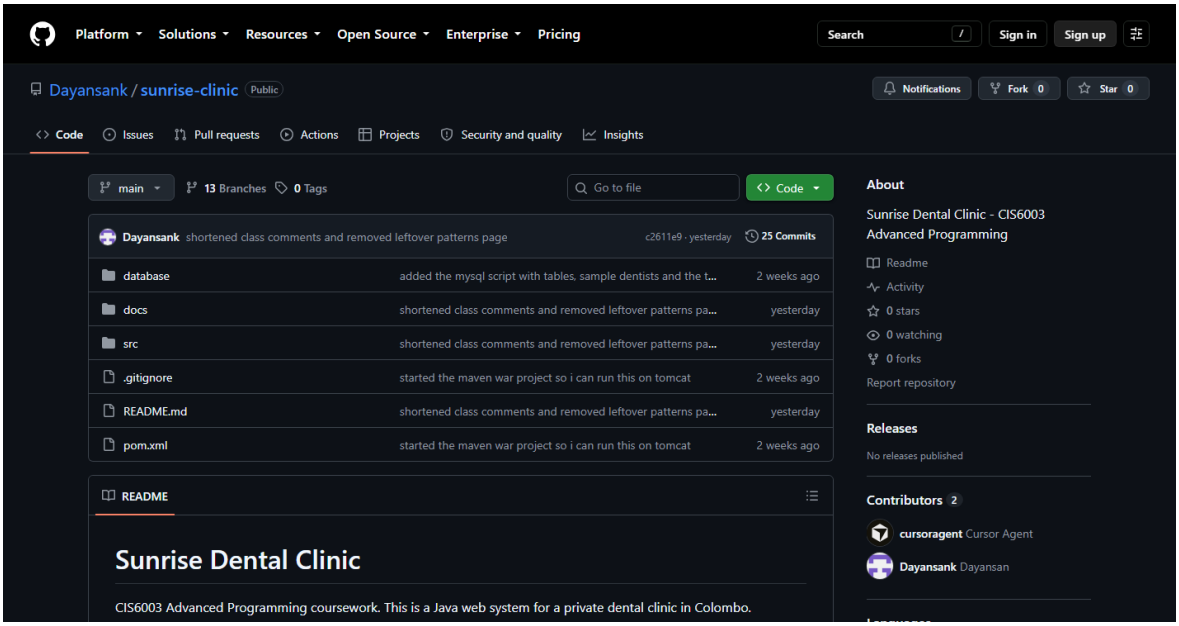

Figure 16: Public GitHub repository

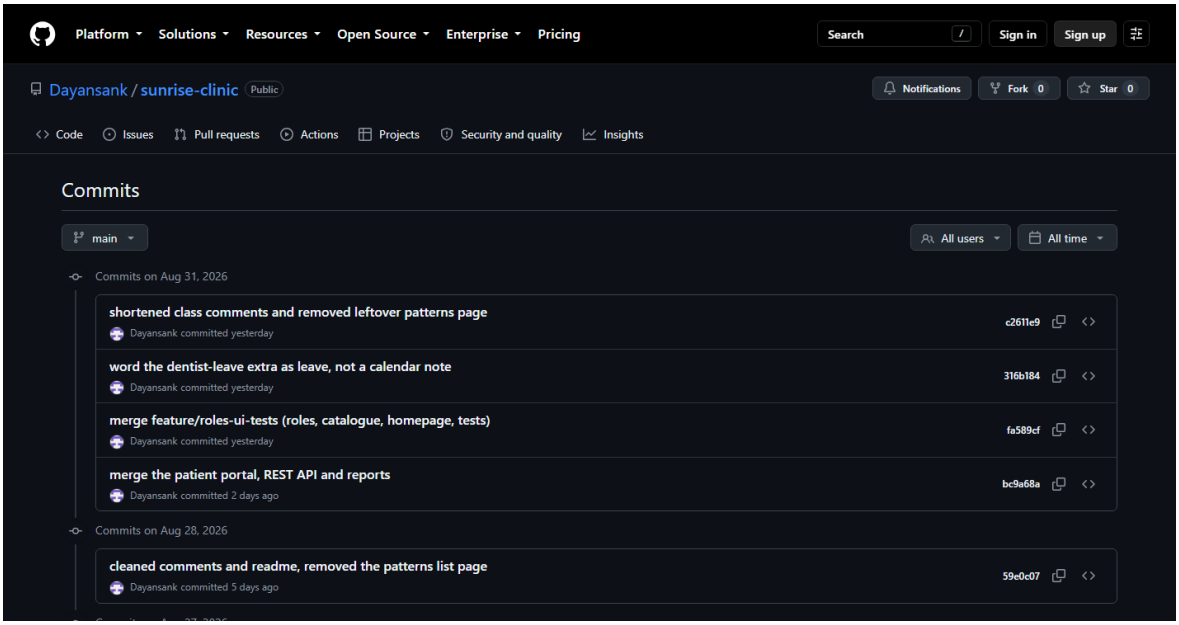

Figure 17: Commit history on main

I run mvn test on JDK 17. README says import the SQL, set db.properties, mvn package, put the WAR on Tomcat. db.properties is not on GitHub.

8 Future implementation

Things I might do later, same clinic. Empty branches are already on GitHub.

- feature/future-sms-email - plug a real SMS and email gateway into NotificationChannel. The include already writes the log rows.
- feature/future-password-reset - forgot password for staff and patients. Login already exists.
- feature/future-card-payment - pay the bill by card after UC4.
- feature/future-reminders - remind the patient the day before a BOOKED visit.
- feature/future-report-pdf - download the reports page as PDF.
- feature/future-dentist-leave - block a dentist who is on leave, still using Check dentist slot.
- feature/future-treatment-notes - short dentist notes on a visit, still searched by APT number.

I will do these after the assignment if I have time.

9 Conclusion

Paper diary was not enough for this clinic. I drew include for validate, slot, find-by-number and notification, and extend for print, QR and the reports date box (Fowler, 2004). Each sequence is on its own page with a short note under it (Sommerville, 2016). Patterns are in Section 5 (Gamma et al., 1995). 45 test cases. Code: https://github.com/Dayansank/sunrise-clinic.

If I had more time I would connect real SMS and make a PDF of the reports. For now the WAR, the diagrams and GitHub are the same system. I also kept the six staff jobs as the main path: login, register, search, bill, help, exit. Extra pages (patient book, reports, staff, catalogue) are still this clinic, they are not a different assignment. Screenshots in Section 4 are from the running Tomcat on this PC.

References

Harvard, hanging indent, A to Z.

Apache Software Foundation (2024) *Apache Tomcat 10 documentation*. Available at: https://tomcat.apache.org/tomcat-10.1-doc/ (Accessed: 2 September 2026).

Beck, K. (2003) *Test-driven development: by example*. Boston: Addison-Wesley.

Booch, G., Rumbaugh, J. and Jacobson, I. (2005) *The unified modeling language user guide*. 2nd edn. Boston: Addison-Wesley.

Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P. and Stal, M. (1996) *Pattern-oriented software architecture: a system of patterns*. Chichester: John Wiley.

Chacon, S. and Straub, B. (2014) *Pro Git*. 2nd edn. New York: Apress. Available at: https://git-scm.com/book/en/v2 (Accessed: 2 September 2026).

Eclipse Foundation (2024) *Jakarta Servlet specification*. Available at: https://jakarta.ee/specifications/servlet/6.0/ (Accessed: 2 September 2026).

Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of database systems*. 7th edn. Harlow: Pearson.

Fielding, R.T. (2000) *Architectural styles and the design of network-based software architectures*. PhD thesis. University of California, Irvine. Available at: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (Accessed: 2 September 2026).

Fowler, M. (2002) *Patterns of enterprise application architecture*. Boston: Addison-Wesley.

Fowler, M. (2004) *UML distilled: a brief guide to the standard object modeling language*. 3rd edn. Boston: Addison-Wesley.

Freeman, E., Robson, E., Bates, B. and Sierra, K. (2004) *Head first design patterns*. Sebastopol, CA: O'Reilly.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1995) *Design patterns: elements of reusable object-oriented software*. Reading, MA: Addison-Wesley.

Horstmann, C.S. (2022) *Core Java*. Volume I. 12th edn. Boston: Pearson.

JUnit Team (2024) *JUnit 5 user guide*. Available at: https://junit.org/junit5/docs/current/user-guide/ (Accessed: 2 September 2026).

Larman, C. (2005) *Applying UML and patterns: an introduction to object-oriented analysis and design and iterative development*. 3rd edn. Upper Saddle River, NJ: Prentice Hall.

Martin, R.C. (2009) *Clean code: a handbook of agile software craftsmanship*. Upper Saddle River, NJ: Prentice Hall.

Oracle (2021) *Java Platform, Standard Edition 17 API specification*. Available at: https://docs.oracle.com/en/java/javase/17/docs/api/index.html (Accessed: 2 September 2026).

Oracle (2024) *MySQL 8.0 reference manual*. Available at: https://dev.mysql.com/doc/refman/8.0/en/ (Accessed: 2 September 2026).

OWASP Foundation (2021) *OWASP Top 10:2021*. Available at: https://owasp.org/Top10/ (Accessed: 2 September 2026).

Sommerville, I. (2016) *Software engineering*. 10th edn. Harlow: Pearson Education.

Appendix A Logins and URLs

- Admin: admin / Admin@123
- Reception: reception / Staff@123
- Patient: kamal@sunrise.lk / Patient@123
- App: http://localhost:8080/sunrise-clinic
- GitHub: https://github.com/Dayansank/sunrise-clinic

Appendix B Diagram sources

All pictures in this Word file are already inserted. UML source files are in docs/uml/. I generated the PNG files with PlantUML. Live screenshots were taken from http://localhost:8080/sunrise-clinic and from https://github.com/Dayansank/sunrise-clinic.